CometBFT 개선 실험 설계

작성일: 2025-01-07 **목적**: Phase 2에서 제안한 CometBFT 개선안의 실험적 구현 설계 **대상 독자**: 합의 알고리즘 연구자, 블록체인 코어 개발자 **선행 문서**: 07-consensus-comparison-and-proposal.md

---

Executive Summary

본 문서는 Phase 2에서 제안한 3가지 CometBFT 개선안 중 **Linear Message Complexity (HotStuff 스타일 BLS 집계)** 를 실험적으로 구현하기 위한 상세 설계를 제시합니다.

실험 목표:

- $O(n^2) \rightarrow O(n)$ 메시지 복잡도 달성
- 100 validators 환경에서 **90%+ 메시지 감소** 검증
- CometBFT 호환성 유지

선택 이유:

1. 구현 복잡도가 상대적으로 낮음
2. 독립적으로 적용 가능 (다른 변경 없이)
3. 측정 가능한 효과가 명확함
4. BLS 라이브러리가 성숙함

목차

1. 실험 개요
2. 이론적 배경
3. 시스템 설계
4. 구현 계획
5. 실험 환경
6. 측정 메트릭
7. 예상 결과
8. 위험 및 제약

---

1. 실험 개요

1.1 문제 정의

현재 CometBFT 투표 흐름:

```
Prevote Phase (n validators):
V1 → V2, V3, V4, ..., Vn  (n-1 messages)
V2 → V1, V3, V4, ..., Vn  (n-1 messages)
...
Vn → V1, V2, V3, ..., Vn-1 (n-1 messages)

Total: n × (n-1) = O(n²) messages
```

100 validators 환경:

- Prevote: 9,900 messages
- Precommit: 9,900 messages
- **Total per block: ~20,000 messages**

1.2 제안하는 해결책

HotStuff 스타일 리더 기반 집계:

```
Prevote Phase:
V1, V2, V3, ..., Vn → Leader  (n messages)
Leader: QC = aggregate(votes)
Leader → V1, V2, V3, ..., Vn  (n messages)

Total: 2n = O(n) messages
```

100 validators 환경:

- Prevote: 200 messages
- Precommit: 200 messages
- **Total per block: ~400 messages (98% 감소!)**

1.3 실험 범위

In Scope:

- Prevote/Precommit 메시지를 리더에게 집중
- BLS 서명 집계
- QC(Quorum Certificate) 브로드캐스트
- 기본적인 리더 장애 처리

Out of Scope (Future Work):

- DAG-based mempool
- Pipelined consensus
- 완전한 view change 프로토콜

---

2. 이론적 배경

2.1 BLS Threshold Signatures

BLS (Boneh-Lynn-Shacham) 서명:

- 짧은 서명 (48 bytes for BLS12-381)
- 서명 집계 가능: $\sigma = \sigma_1 + \sigma_2 + \ldots + \sigma_n$
- 집계된 서명 검증: O(1) pairing 연산

동작 원리:

```
개별 서명:
σᵢ = Sign(skᵢ, message)

서명 집계:
σ = σ₁ + σ₂ + ... + σₖ  (k ≥ 2f+1)

공개키 집계:
PK = pk₁ + pk₂ + ... + pkₖ

검증:
Verify(PK, message, σ) = true
```

2.2 Quorum Certificate (QC)

QC 구조:

```go
type QuorumCertificate struct {
    Height       int64
    Round        int32
    BlockHash    []byte
    Signature    []byte      // 집계된 BLS 서명
    Signers      *BitArray   // 서명자 비트맵
}
```

QC 검증:

1. Signers 비트맵에서 참여자 추출
2. 참여자 수 ≥ 2f+1 확인
3. 참여자 공개키 집계
4. 집계된 서명 검증

2.3 안전성 증명 (Informal)

Safety Invariant:

> 동일 (height, round)에서 두 개의 다른 블록에 대한 QC가 생성되지 않는다.

증명 스케치:

1. QC 생성에 2f+1 서명 필요
2. Byzantine validators 최대 f
3. 두 개의 다른 QC는 각각 2f+1 서명 필요
4. 교집합: (2f+1) + (2f+1) - n = 2f+1 + 2f+1 - 3f - 1 = f+1
5. 최소 f+1 honest validators가 두 블록 모두 서명해야 함
6. Honest validators는 같은 (height, round)에서 하나만 서명
7. **모순** → 두 개의 다른 QC 불가능

Liveness:

- 리더 장애 시 타임아웃 후 round 증가
- 새 리더가 이전 QC를 포함하여 제안
- GST(Global Stabilization Time) 이후 진행 보장

---

3. 시스템 설계

3.1 아키텍처 개요

```
┌─────────────────────────────────────────────────────────────┐
│                  Modified CometBFT Architecture              │
├─────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────────────────────────────────────────────┐       │
│  │                  Consensus Module                 │   │   │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐       │   │   │
│  │  │ Propose  │──│ Prevote  │──│ Precommit │      │   │   │
│  │  └──────────┘  └──────────┘  └──────────┘       │   │   │
│  │                     │              │             │   │   │
│  │                     ▼              ▼             │   │   │
│  │                ┌──────────────────────┐         │   │   │
│  │                │   Vote Aggregator (NEW)  │     │   │   │
│  │                │  ┌──────────────────┐  │       │   │   │
│  │                │  │ BLS Threshold    │  │       │   │   │
│  │                │  │ Signature Module │  │       │   │   │
│  │                │  └──────────────────┘  │       │   │   │
│  │                │                        │       │   │   │
│  │                │  ┌──────────────────┐  │       │   │   │
│  │                │  │ QC Builder       │  │       │   │   │
│  │                │  └──────────────────┘  │       │   │   │
│  │                └──────────────────────┘         │   │   │
│  └──────────────────────────────────────────────────┘       │
│                                                              │
│  ┌──────────────────────────────────────────────────┐       │
│  │                    P2P Layer                     │   │   │
│  │  ┌──────────────────────────────────────────┐    │   │   │
│  │  │ Modified Reactor: Leader-directed messages │  │   │   │
│  │  └──────────────────────────────────────────┘    │   │   │
│  └──────────────────────────────────────────────────┘       │
│                                                              │
└─────────────────────────────────────────────────────────────┘
```

3.2 새로운 컴포넌트

3.2.1 BLS Module

```go
// internal/bls/bls.go

package bls

import (
    "github.com/herumi/bls-eth-go-binary/bls"
)

// Initialize BLS library
func init() {
    if err := bls.Init(bls.BLS12_381); err != nil {
        panic(err)
    }
}

// KeyPair represents a BLS key pair
type KeyPair struct {
    SecretKey *bls.SecretKey
    PublicKey *bls.PublicKey
}

// GenerateKeyPair creates a new BLS key pair
func GenerateKeyPair() *KeyPair {
    sk := new(bls.SecretKey)
    sk.SetByCSPRNG()
    pk := sk.GetPublicKey()
    return &KeyPair{SecretKey: sk, PublicKey: pk}
}

// Sign creates a BLS signature
func Sign(sk *bls.SecretKey, message []byte) *bls.Sign {
    sig := sk.SignByte(message)
    return sig
}

// Verify checks a BLS signature
func Verify(pk *bls.PublicKey, message []byte, sig *bls.Sign) bool {
    return sig.VerifyByte(pk, message)
}

// AggregateSignatures combines multiple signatures
func AggregateSignatures(sigs []*bls.Sign) *bls.Sign {
    if len(sigs) == 0 {
        return nil
    }
    agg := new(bls.Sign)
    agg.Aggregate(sigs)
    return agg
}

// AggregatePublicKeys combines multiple public keys
func AggregatePublicKeys(pks []*bls.PublicKey) *bls.PublicKey {
    if len(pks) == 0 {
        return nil
    }
    agg := new(bls.PublicKey)
    agg.Aggregate(pks)
    return agg
}

// VerifyAggregated verifies an aggregated signature
func VerifyAggregated(aggregatedPK *bls.PublicKey, message []byte, aggregatedSig *bls.Sign) bool {
    return aggregatedSig.VerifyByte(aggregatedPK, message)
}
```

3.2.2 Vote Aggregator

```go
// internal/consensus/vote_aggregator.go

package consensus

import (
    "sync"

    "github.com/cometbft/cometbft/internal/bls"
    "github.com/cometbft/cometbft/types"
)

// VoteAggregator collects and aggregates votes for a height/round
type VoteAggregator struct {
    height int64
    round  int32

    // Validator set for this height
    validators *types.ValidatorSet

    // Collected votes (prevotes and precommits)
    prevotes   map[int]*AggregatedVote  // round -> votes
    precommits map[int]*AggregatedVote

    // BLS public keys (indexed by validator index)
    pubKeys []*bls.PublicKey

    mu sync.RWMutex
}

// AggregatedVote holds votes for a specific block
type AggregatedVote struct {
    BlockHash  []byte
    Signatures []*bls.Sign
    Signers    *types.BitArray
    Count      int
}

// NewVoteAggregator creates a new vote aggregator
func NewVoteAggregator(height int64, validators *types.ValidatorSet, pubKeys []*bls.PublicKey) *VoteAggregator {
    return &VoteAggregator{
        height:     height,
        validators: validators,
        prevotes:   make(map[int]*AggregatedVote),
        precommits: make(map[int]*AggregatedVote),
        pubKeys:    pubKeys,
    }
}

// AddPrevote adds a prevote to the aggregator
func (va *VoteAggregator) AddPrevote(round int32, validatorIdx int, blockHash []byte, sig *bls.Sign) error {
    va.mu.Lock()
    defer va.mu.Unlock()

    roundKey := int(round)
    if va.prevotes[roundKey] == nil {
        va.prevotes[roundKey] = &AggregatedVote{
            BlockHash:  blockHash,
            Signatures: make([]*bls.Sign, 0),
            Signers:    types.NewBitArray(va.validators.Size()),
        }
    }

    av := va.prevotes[roundKey]

    // Check if already voted
    if av.Signers.GetIndex(validatorIdx) {
        return ErrDuplicateVote
    }

    // Verify signature
    message := va.voteMessage(round, types.PrevoteType, blockHash)
    if !bls.Verify(va.pubKeys[validatorIdx], message, sig) {
        return ErrInvalidSignature
    }

    // Add vote
```

```go
    av.Signatures = append(av.Signatures, sig)
    av.Signers.SetIndex(validatorIdx, true)
    av.Count++

    return nil
}

// BuildPrevoteQC creates a QC if we have enough votes
func (va *VoteAggregator) BuildPrevoteQC(round int32) (*QuorumCertificate, error) {
    va.mu.RLock()
    defer va.mu.RUnlock()

    av := va.prevotes[int(round)]
    if av == nil {
        return nil, ErrNoVotes
    }

    // Check quorum
    threshold := va.validators.TwoThirdsMajority()
    if av.Count < threshold {
        return nil, ErrNotEnoughVotes
    }

    // Aggregate signatures
    aggregatedSig := bls.AggregateSignatures(av.Signatures)

    // Aggregate public keys of signers
    var signerPKs []*bls.PublicKey
    for i := 0; i < va.validators.Size(); i++ {
        if av.Signers.GetIndex(i) {
            signerPKs = append(signerPKs, va.pubKeys[i])
        }
    }
    aggregatedPK := bls.AggregatePublicKeys(signerPKs)

    // Verify aggregated signature
    message := va.voteMessage(round, types.PrevoteType, av.BlockHash)
    if !bls.VerifyAggregated(aggregatedPK, message, aggregatedSig) {
        return nil, ErrAggregationFailed
    }

    return &QuorumCertificate{
        Type:      types.PrevoteType,
        Height:    va.height,
        Round:     round,
        BlockHash: av.BlockHash,
        Signature: aggregatedSig.Serialize(),
        Signers:   av.Signers,
    }, nil
}

// Helper to construct vote message
func (va *VoteAggregator) voteMessage(round int32, voteType types.SignedMsgType, blockHash []byte) []byte {
    // Canonical encoding of vote data
    // Similar to types.VoteSignBytes
    return types.VoteSignBytes("chain-id", &types.Vote{
        Type:      voteType,
        Height:    va.height,
        Round:     round,
        BlockID:   types.BlockID{Hash: blockHash},
    })
}
```

3.2.3 QuorumCertificate

```go
// types/qc.go

package types

import (
    "github.com/cometbft/cometbft/internal/bls"
)

// QuorumCertificate represents an aggregated vote certificate
type QuorumCertificate struct {
    Type      SignedMsgType
    Height    int64
    Round     int32
    BlockHash []byte
    Signature []byte    // Aggregated BLS signature
    Signers   *BitArray // Bitmap of signers
}

// VerifyQC verifies the quorum certificate
func (qc *QuorumCertificate) Verify(validators *ValidatorSet, pubKeys []*bls.PublicKey, chainID string) bool {
    // Check signer count
    signerCount := qc.Signers.CountTrue()
    if signerCount < validators.TwoThirdsMajority() {
        return false
    }

    // Collect signer public keys
    var signerPKs []*bls.PublicKey
    for i := 0; i < validators.Size(); i++ {
        if qc.Signers.GetIndex(i) {
            signerPKs = append(signerPKs, pubKeys[i])
        }
    }

    // Aggregate public keys
    aggregatedPK := bls.AggregatePublicKeys(signerPKs)

    // Reconstruct vote message
    message := VoteSignBytes(chainID, &Vote{
        Type:    qc.Type,
        Height:  qc.Height,
        Round:   qc.Round,
        BlockID: BlockID{Hash: qc.BlockHash},
    })

    // Verify aggregated signature
    sig := new(bls.Sign)
    if err := sig.Deserialize(qc.Signature); err != nil {
        return false
    }

    return bls.VerifyAggregated(aggregatedPK, message, sig)
}

// ToProto converts to protobuf
func (qc *QuorumCertificate) ToProto() *cmtproto.QuorumCertificate {
    return &cmtproto.QuorumCertificate{
        Type:      cmtproto.SignedMsgType(qc.Type),
        Height:    qc.Height,
        Round:     qc.Round,
        BlockHash: qc.BlockHash,
        Signature: qc.Signature,
        Signers:   qc.Signers.ToProto(),
    }
}
```

3.3 수정된 합의 흐름

3.3.1 Prevote Phase (수정)

기존:

```
enterPrevote()
    │
    ├─> signAddVote()
    │     └─> broadcastVote()  // 모든 피어에게
    │
    └─> 모든 노드가 모든 투표 수집
```

수정:

```
enterPrevote()
    │
    ├─> isLeader()?
    │     │
    │     ├ Yes: startVoteCollection()
    │     │        └─> 타임아웃 설정, 투표 대기
    │     │
    │     └ No: sendVoteToLeader()
    │             └─> 리더에게만 투표 전송
    │
    └─> onReceiveVote() (리더만)
          │
          ├─> aggregator.AddPrevote()
          │
          └─> hasQuorum()?
                │
                ├ Yes: buildAndBroadcastQC()
                │
                └ No: continue waiting
```

3.3.2 QC 수신 처리

```
onReceiveQC()
    │
    ├─> verifyQC()
    │     ├ 서명 검증
    │     ├ quorum 확인
    │     └ height/round 확인
    │
    ├─> valid?
    │     │
    │     ├ Yes: enterNextPhase()
    │     │        └─> Prevote QC → enterPrecommit
    │     │            Precommit QC → enterCommit
    │     │
    │     └ No: log warning, ignore
    │
    └─> storeQC() (consensus 재시작용)
```

3.4 프로토콜 메시지

새로운 메시지 타입:

```
// proto/cometbft/consensus/v1/types.proto

// VoteToLeader is sent from validators to the round leader
message VoteToLeader {
    SignedMsgType type = 1;
    int64 height = 2;
    int32 round = 3;
    bytes block_hash = 4;
    bytes bls_signature = 5;
    int32 validator_index = 6;
}

// QuorumCertificateBroadcast is sent from leader to all validators
message QuorumCertificateBroadcast {
    SignedMsgType type = 1;
    int64 height = 2;
    int32 round = 3;
    bytes block_hash = 4;
    bytes aggregated_signature = 5;
    BitArray signers = 6;
}
```

---

4. 구현 계획

4.1 Phase별 구현

```
┌─────────────────────────────────────────────────────────────┐
│                    Implementation Phases                    │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  Phase 1 (Week 1-2): BLS 인프라                              │
│  ├─ BLS 라이브러리 통합 (herumi/bls-eth-go-binary)            │
│  ├─ KeyPair 생성 및 관리                                     │
│  ├─ 서명/검증 래퍼 함수                                       │
│  └─ 단위 테스트                                              │
│                                                             │
│  Phase 2 (Week 3-4): Vote Aggregator                        │
│  ├─ VoteAggregator 구현                                      │
│  ├─ QC 빌더 구현                                             │
│  ├─ QC 검증 로직                                             │
│  └─ 단위 테스트                                              │
│                                                             │
│  Phase 3 (Week 5-6): Consensus 통합                          │
│  ├─ enterPrevote/enterPrecommit 수정                         │
│  ├─ 리더 기반 메시지 라우팅                                    │
│  ├─ QC 브로드캐스트                                           │
│  └─ 통합 테스트                                              │
│                                                             │
│  Phase 4 (Week 7-8): 테스트 및 벤치마크                        │
│  ├─ 4-node 테스트넷 구축                                      │
│  ├─ 메시지 수 측정                                            │
│  ├─ 레이턴시 측정                                             │
│  └─ 결과 분석 및 문서화                                       │
│                                                             │
└─────────────────────────────────────────────────────────────┘

총 기간: 8주
```

4.2 파일 변경 목록

새 파일:

```
internal/
├ bls/
│   ├ bls.go           # BLS 래퍼
│   └ bls_test.go
├ consensus/
│   ├ vote_aggregator.go
│   └ vote_aggregator_test.go
types/
├ qc.go                # QuorumCertificate
└ qc_test.go
proto/
└ cometbft/consensus/v1/
    └ types.proto      # 새 메시지 타입
```

수정 파일:

```
internal/consensus/
├ state.go             # 합의 상태 머신
├ reactor.go           # P2P 메시지 처리
└ ...
types/
├ vote.go              # BLS 서명 필드 추가
└ ...
```

4.3 의존성

외부 라이브러리:

```go
// go.mod
require (
    github.com/herumi/bls-eth-go-binary v1.28.2
)
```

빌드 요구사항:

- CGO 활성화 (BLS 라이브러리)
- 또는 pure Go 대안: `github.com/kilic/bls12-381`

---

5. 실험 환경

5.1 테스트넷 구성

최소 구성 (개발/디버깅):

```
4 validators:
├ V1 (leader 0): 8GB RAM, 4 CPU
├ V2:            8GB RAM, 4 CPU
├ V3:            8GB RAM, 4 CPU
└ V4:            8GB RAM, 4 CPU

네트워크: 같은 호스트 (localhost)
지연: 인위적 지연 없음
```

중간 구성 (성능 테스트):

```
10 validators:
├ 10× VMs: 8GB RAM, 4 CPU
├ 같은 데이터센터
└ 네트워크 지연: 1-5ms
```

대규모 구성 (확장성 테스트):

```
100 validators:
├ 100× VMs: 4GB RAM, 2 CPU
├ 여러 지역
└ 네트워크 지연: 10-100ms
```

5.2 테스트 시나리오

시나리오 1: 정상 동작

- 모든 validators 정상
- 블록 생성 및 합의
- 메시지 수 측정

시나리오 2: 리더 장애

- 라운드 리더 크래시
- 타임아웃 후 라운드 변경
- 새 리더로 진행

시나리오 3: 네트워크 파티션

- 일시적 네트워크 분리
- 복구 후 합의 재개

시나리오 4: Byzantine 노드

- f개의 악의적 노드
- 이중 투표 시도
- 시스템 안전성 검증

5.3 Docker 구성

```yaml
# docker-compose.yml

version: '3.8'

services:
  validator1:
    build: .
    command: ["cometbft", "start", "--proxy_app=kvstore"]
    volumes:
      - ./config/validator1:/cometbft/config
    networks:
      - testnet
    environment:
      - VALIDATOR_INDEX=0
      - IS_LEADER_FIRST_ROUND=true

  validator2:
    build: .
    command: ["cometbft", "start", "--proxy_app=kvstore"]
    volumes:
      - ./config/validator2:/cometbft/config
    networks:
      - testnet
    environment:
      - VALIDATOR_INDEX=1

  validator3:
    build: .
    command: ["cometbft", "start", "--proxy_app=kvstore"]
    volumes:
      - ./config/validator3:/cometbft/config
    networks:
      - testnet
    environment:
      - VALIDATOR_INDEX=2

  validator4:
    build: .
    command: ["cometbft", "start", "--proxy_app=kvstore"]
    volumes:
      - ./config/validator4:/cometbft/config
    networks:
      - testnet
    environment:
      - VALIDATOR_INDEX=3

networks:
  testnet:
    driver: bridge
```

---

6. 측정 메트릭

6.1 주요 메트릭

메트릭	설명	측정 방법
Messages/Block	블록당 메시지 수	P2P 레이어 카운터
Latency (ms)	합의 레이턴시	Propose → Commit 시간
Throughput (TPS)	트랜잭션 처리량	tx/sec
Bandwidth (KB/s)	네트워크 대역폭	P2P 바이트 카운터
CPU Usage (%)	CPU 사용량	시스템 메트릭
Memory (MB)	메모리 사용량	시스템 메트릭

6.2 메트릭 수집

```go
// metrics/consensus_metrics.go

package metrics

import (
    "github.com/prometheus/client_golang/prometheus"
)

var (
    // Message counts
    PrevoteMessagesSent = prometheus.NewCounter(prometheus.CounterOpts{
        Name: "consensus_prevote_messages_sent",
        Help: "Total prevote messages sent",
    })

    QCBroadcasts = prometheus.NewCounter(prometheus.CounterOpts{
        Name: "consensus_qc_broadcasts",
        Help: "Total QC broadcasts",
    })

    // Latency
    RoundDuration = prometheus.NewHistogram(prometheus.HistogramOpts{
        Name:    "consensus_round_duration_seconds",
        Help:    "Duration of consensus rounds",
        Buckets: prometheus.ExponentialBuckets(0.01, 2, 15),
    })

    // Aggregation stats
    VotesAggregated = prometheus.NewHistogram(prometheus.HistogramOpts{
        Name:    "consensus_votes_aggregated",
        Help:    "Number of votes aggregated per QC",
        Buckets: prometheus.LinearBuckets(1, 10, 20),
    })

    AggregationLatency = prometheus.NewHistogram(prometheus.HistogramOpts{
        Name:    "consensus_aggregation_latency_ms",
        Help:    "Time to aggregate votes",
        Buckets: prometheus.ExponentialBuckets(0.1, 2, 12),
    })
)
```

6.3 데이터 수집 스크립트

```bash
#!/bin/bash
# collect_metrics.sh

# Run for 100 blocks
BLOCKS=100
OUTPUT_DIR="./results/$(date +%Y%m%d_%H%M%S)"
mkdir -p $OUTPUT_DIR

# Start metrics collection
echo "Collecting metrics for $BLOCKS blocks..."

for i in $(seq 1 4); do
    docker logs validator$i 2>&1 | grep -E "(messages|latency|block)" > $OUTPUT_DIR/validator$i.log &
done

# Wait for blocks
sleep $((BLOCKS * 2))  # ~2 seconds per block

# Collect Prometheus metrics
curl -s http://localhost:9090/api/v1/query?query=consensus_prevote_messages_sent > $OUTPUT_DIR/messages.json
curl -s http://localhost:9090/api/v1/query?query=consensus_round_duration_seconds > $OUTPUT_DIR/latency.json

# Analyze
python3 analyze_metrics.py $OUTPUT_DIR
```

---

7. 예상 결과

7.1 메시지 복잡도

이론적 예측:

Validators	기존 ($O(n^2)$)	개선 ($O(n)$)	감소율
4	24	8	67%
10	180	20	89%
50	4,900	100	98%
100	19,800	200	99%

7.2 레이턴시

예상 변화:

```
4 validators (ideal network):
├ 기존:  ~100ms (3 round-trips)
└ 개선:  ~80ms  (2 round-trips to leader + 1 broadcast)
         개선: 20%

100 validators (realistic network):
├ 기존:  ~500ms (메시지 폭주로 인한 지연)
└ 개선:  ~200ms (리더 집중으로 효율화)
         개선: 60%
```

7.3 처리량

예상 TPS 변화:

Validators	기존 TPS	개선 TPS	증가율
4	1,000	1,200	20%
10	800	1,100	38%
50	400	800	100%
100	200	600	200%

7.4 성공 기준

기준	목표
메시지 감소	90%+ (100 validators)
레이턴시	기존 대비 악화 없음
정확성	모든 블록 정상 합의
안전성	Byzantine 노드에서 안전

---

8. 위험 및 제약

8.1 기술적 위험

위험	확률	영향	완화 방안
BLS 라이브러리 버그	낮음	높음	검증된 라이브러리 사용, 광범위 테스트
리더 병목	중간	중간	타임아웃 조정, 비동기 처리
CGO 호환성	중간	중간	Pure Go 대안 준비
합의 안전성 문제	낮음	매우 높음	형식적 검증, 철저한 테스트

8.2 제약 사항

1. **호환성**: 기존 CometBFT 네트워크와 호환 불가 (별도 테스트넷 필요)
2. **키 관리**: BLS 키 별도 관리 필요
3. **업그레이드**: Hard fork 필요
4. **성숙도**: 실험적 구현 (프로덕션 미권장)

8.3 대안 계획

BLS 라이브러리 문제 시:

- herumi → kilic 라이브러리로 전환
- Pure Go로 성능 저하 감수

리더 병목 심각 시:

- 다중 리더 구조 도입 (복잡도 증가)
- 메시지 압축 추가

실험 실패 시:

- 결과 문서화
- 교훈 정리
- 대안 접근법 제안

결론

요약

본 실험은 CometBFT의 $O(n^2)$ 메시지 복잡도를 $O(n)$으로 개선하기 위해 HotStuff 스타일의 BLS 서명 집계를 구현합니다.

핵심 기여:

1. BLS threshold signatures 통합
2. 리더 기반 투표 집계
3. Quorum Certificate 메커니즘
4. 성능 측정 및 분석

예상 성과:

- 100 validators 환경에서 **99% 메시지 감소**
- 처리량 **2-3배 향상**
- 합의 안전성 유지

다음 단계

1. **즉시**: BLS 라이브러리 평가 및 통합
2. **2주 내**: VoteAggregator PoC 완료
3. **4주 내**: 4-node 테스트넷 구동
4. **8주 내**: 전체 실험 완료 및 결과 문서화

장기 로드맵

이 실험이 성공하면:

1. **PR 제출**: CometBFT 또는 Cosmos SDK에 기여
2. **확장**: DAG-based mempool 실험
3. **통합**: Cosmos EVM과 통합 테스트

작성 완료: 2025-01-07 **총 페이지**: ~20페이지 **분석 기반**: CometBFT 소스코드, HotStuff 논문, BLS 표준 **다음 문서**: `04-experiment-results.md` (실험 결과 - 구현 후 작성)